driftwatch: Project Brief

Did the model change, or did the users? Label-free monitoring of a deployed language model.

Dhruv Parmar · Pronnati Tapaswi · Mahak Khurdia · Aditya Jana
Dhirubhai Ambani University, Gandhinagar
23 October 2026

The project on one page

The problem. A language model behind an API can change without telling anyone: a new version under the same name, a cheaper model, a lower-precision build, a dropped setting. Nothing crashes, and production traffic has no labels, so the change has to be noticed from the answers alone.

Why it is hard. Users change too. When the mix of questions shifts, every statistic of the answers moves although the model is the same. A monitor that compares live traffic with a fixed healthy reference alarmed on 98–100% of such harmless shifts.

What we built. Four monitors that differ in what they compare a window of live answers with. The two that work **re-ask a reference model the same live questions** and compare answer with answer, so a change of topic moves both sides alike. The last of them, the **served-answer test**, needs only the one answer each user actually received.

What we found:

- **Real updates are caught on both vendors.** The served-answer test catches the local faults in 99–100% of streams, DeepSeek's real update in 95%, the same update on real chatbot traffic in 94%, and Qwen's real update in 96.5%.
- **No false alarms, and topic shifts are ignored.** It raised no false alarm in 87,716 healthy windows and alarmed on 0–1% of topic shifts.
- **The proof.** A short proof shows its score is centred on zero for every question when nothing changed. That is why topic shifts cannot move it.
- **Calibration must span days.** Hosted models drift from day to day, so calibration has to use several healthy rounds.
- **A blind test confirms it.** Run frozen on five changes it had never seen, it caught four in 98–100% of streams with no false alarm, and missed answers cut short, which lie beyond the 1,000 characters it compares.
- **An alarm means change, not harm.** Two of the changes we caught made the model *better*, and a small graded check after each alarm told the two apart without a single wrong verdict.

How we kept it honest. Every study was pre-registered in a committed file before its data existed, every threshold was frozen before any fault answer was read, and every later change is a dated amendment. Every study was replayed five times. Total hosted spend: $26.11.

How to read this

- **Part I** is the problem.
- **Part II** is what we built and why it works.
- **Part III** covers every experiment, each set out as *what we did*, *what came out*, and *what it means*.
- **Part IV** is what the results mean, and their limits.
- **Part V** is the practical side: cost, how to run it, the papers, and what is left.

All numbers are from the primary replay (§24). The ACL paper has the same numbers with intervals; if the two ever disagree, the paper and the records in `data/` win.

PART I: THE PROBLEM

1. The situation

A company puts a language model behind its product. It answers maths homework, summarises articles, sorts support tickets. It works for months.

Then something changes that nobody announced. During this project that happened for real: on 17 October 2026 DeepSeek replaced V4 Flash with V4.1 Flash behind the same API name. Elsewhere a serving team moves to a smaller model to save money, ships a quantised build, edits the system prompt, or loses the temperature setting in a config change.

None of this throws an error. Latency is normal, error rates are normal, the answers read fluently. The model degrades without crashing, and it can stay that way for weeks. Knowing that quality moved normally needs the right answers, and nobody labels live traffic. A benchmark run on a schedule helps, but it tests a fixed question set, not what users actually ask.

So the question is:

> *Can a monitor that sees only the questions and answers of live traffic, never a label, tell that the model changed, at a false-alarm rate fixed in advance?*

2. The hard part: users change too

The obvious design records what healthy answers look like (their length, how consistent they are, what they are about) and alarms when a window of live answers drifts away from that.

It fails, and finding out how was our first real result. A week of maths homework produces short answers; a week of "explain like I'm five" produces long ones. The model is the same, yet every statistic of its output has moved. A monitor that compares live traffic with a fixed reference cannot tell "the model answers differently" from "users ask different things", and an operator switches it off within a week.

The real job is therefore to **separate a change of model from a change of users**.

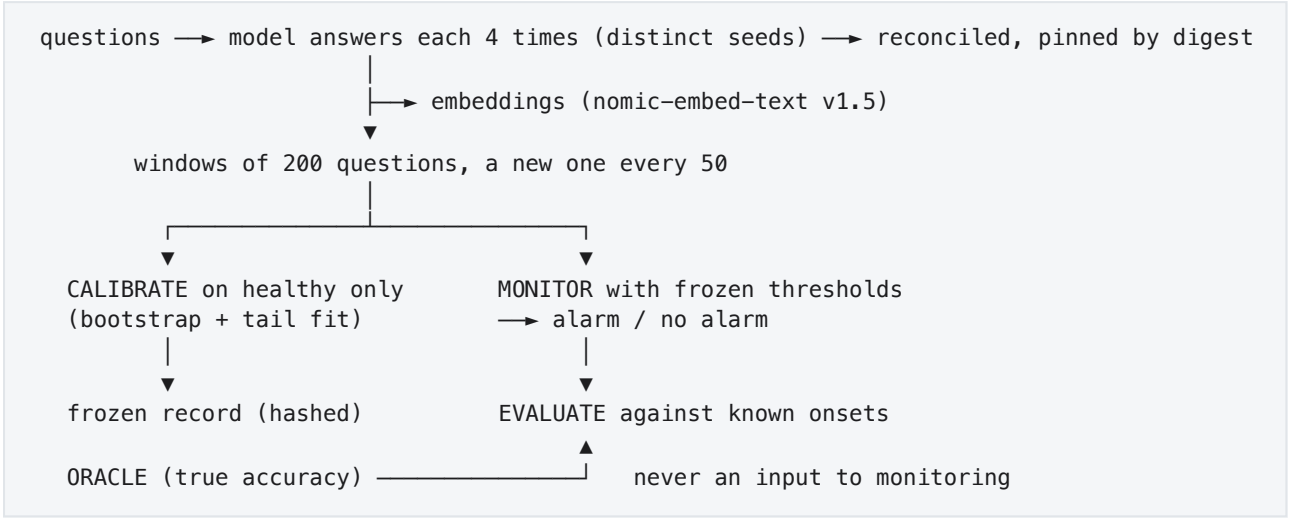
3. Why this is research, not plumbing

Most drift detectors are tested on drift somebody already noticed, with no ground truth about when it began and rarely any test of whether they stay quiet when they should. We control both:

- **A known onset.** Every change starts at a known window, so latency and early alarms can be measured.
- **Harmless changes on purpose.** Topic shifts and reruns of the healthy model are built in, so false alarms can be counted where a naive monitor would fire.
- **Frozen thresholds.** Every threshold is set on healthy data and frozen before any fault answer is read.
- **Pre-registration.** Every study was committed as a config file before its data existed.

PART II: WHAT WE BUILT

4. The pipeline



The monitor never sees the oracle. A test walks the import graph and fails the build if any monitoring code imports the grader, even indirectly.

5. Windows

One answer says nothing; drift lives in distributions. Everything is scored over **windows of 200 questions, with a new window every 50**, so neighbours overlap by

1. On benchmark traffic windows keep the usual mix of task types, so a statistic

that exists on one task only (intent labels, say) always has data.

6. The four monitors

They differ in **what a window of live answers is compared with**.

monitor	compared with	task labels	re-asks a reference	production answers per question
pooled (v1)	a fixed healthy reference	no	no	4
per-family (v3)	a fixed reference, one task type at a time	yes	no	4
paired	the reference's answers to the same questions	no	yes	4
served answer	the reference's answers to the same questions	no	yes	**1**

Pooled (v1). Five label-free statistics, each comparing the window with healthy answers:

- *answer length*, as a Kolmogorov–Smirnov statistic;

- *agreement* among the four sampled answers to a question;
- *lexical diversity*, as a length-normalised type-token ratio (MATTR);
- *label entropy* of predicted intents on the classification task;
- *embedding shift* of the window's answers away from the healthy ones.

The first design had seven signals. Refusal rate and JSON validity never moved on healthy data (the model essentially never refused, and its JSON always parsed), so calibration set them aside as unmonitored.

Per-family (v3). The same statistics, computed within one task type at a time against that type's own healthy answers, then averaged with each type's weight capped at its usual share of traffic. More maths in a window no longer looks like shorter answers. But it needs the task type of every question, which real traffic does not carry. (v2 was the same without the cap; it failed an unseen test, which is why v3 exists. Both are recorded as designs made after v1 failed.)

Paired. Keep the old model available (a kept checkpoint or a pinned API version) and **ask it the window's own questions again**, four times each. Compare the live and reference answers question by question:

- the length distributions (KS);
- the gap in agreement between samples;
- one minus the cosine similarity of their mean embeddings.

If only the topics changed, both sides change alike and nothing moves. Lexical diversity was dropped here because it drifts between runs of the same model (§13).

Served answer. Production serves one answer per request. Drawing three more costs money, and they are not what users received. For a question with served answer c and four reference answers $r_1 \ldots r_4$ it computes

```
s = (average similarity between pairs of reference answers)
  − (average similarity between c and each reference answer)
```

Similarity is the share of matching characters among the first 1,000, the kernel Model Equality Testing uses (Gao, Liang and Guestrin, ICLR 2025). The window statistic is the mean of s, and as there is one statistic, it gets the whole false-alarm budget.

7. What the served score measures

For one question, call K_{rr} the expected similarity between two reference answers, K_{cc} between two answers of the served model, and K_{cr} between one of each. A three-line proof (Appendix F of the paper) gives

$$E[s] = K_{rr} - K_{cr} = \tfrac{1}{2}\cdot \mathrm{MMD}^2 + \tfrac{1}{2}\cdot(K_{rr} - K_{cc})$$

where MMD^2 is the per-question distance that Model Equality Testing estimates. Three consequences follow, and the experiments bear out each one:

1. **No change means zero, question by question.** If the model did not change, all three similarities are equal and the expected score is zero *for every question*. No mix of questions can move the window mean, so a topic shift cannot set the test off. This is the mechanism behind the whole approach.
2. **A change shows up in two parts.** The score rises by half of MET's distance plus half of how much less alike the new model's answers are to each other. More varied answers (a higher temperature) are easier to see.
3. **There is a blind spot.** If a change makes answers *more* alike than the old model's, the second term is negative and cancels part of the first. MET has no such blind spot, but it needs several production answers per question.

8. Calibration

The budget is **one false alarm per 1,000 healthy windows (0.1%)**, counted family-wise: the chance that *any* statistic fires on a healthy window. Three traps would each inflate every number we publish:

1. **Several statistics share one budget.** Each gets a Šidák share, $\alpha' = 1 - (1 - 0.001)^{(1/m)}$. Our statistics are correlated, which makes this conservative, not wrong.
2. **There are too few real windows.** The calibration questions give about 46 distinct windows, too few to read a 99.9th percentile. We draw 20,000 bootstrap windows under "no change".
3. **The tail is still thin.** Only a handful of draws sit beyond the threshold, so we fit a generalised Pareto tail and read the threshold off the fit. One fit came out degenerate on tied values (a threshold of 2×10^{16}). Such fits now fall back to probability-weighted moments, and that record was withdrawn and refitted before any evaluation used it.

For the re-asking monitors, "no change" means two healthy answer sets for the same questions. How many such sets you need became a main finding (§13).

9. Freezing and pre-registration

Calibrated thresholds go into a frozen record with the git commit, a content hash of the exact calibration rows, the model ids and digests, and the seeds. Every evaluation re-verifies the record and refuses to run on a single changed row. Re-asking records are committed to git before they are evaluated.

Every study was pre-registered: a config file in `src/config/` with its conditions, analysis and pass/fail criteria, committed before the answers that judge it existed. Changes made later are dated amendments that say what changed, why, and what had been seen by then. Some are awkward:

- a design changed after an earlier one failed;
- a study resumed after its partners' results were read;
- a control that turned out to be impossible.

They are recorded anyway. That record is what makes the rest believable.

10. The oracle

After detection is fixed, the oracle grades answers against ground truth:

- **exact or fuzzy match** for maths (GSM8K), trivia (TriviaQA) and reading comprehension (SQuAD 2.0);
- **label accuracy** for intent classification (Banking77);
- **strict schema match** for JSON extraction (jsonx, which we built ourselves).

Summaries (XSum) and explanations (ELI5) are not graded. The oracle only answers "how much did this change really cost?".

PART III: THE EXPERIMENTS

11. Setup

Questions. 5,000 over seven task types: maths, trivia, reading comprehension, intent classification, JSON extraction, one-sentence summaries and long explanations. They are split 50/20/30 into calibration (2,500), held-out (1,000, used only to count false alarms) and fault (1,500) questions.

The healthy model. `gemma4:26b-a4b` (25.8B mixture-of-experts, about 4B active), 4-bit, run locally through Ollama and pinned by digest. Every question is answered four times with distinct seeds, at temperature 0.7 and up to 512 tokens.

Local faults, all on the same 1,500 fault questions:

	what changes	accuracy change*
F1	a much smaller sibling, `gemma4:e2b` (4.6B)	−11.3
F2	a smaller sibling, `gemma4:e4b` (8.0B)	−5.7
F3	two lines added to the system prompt, a plausible careless edit	−0.5
F5	an older generation, `gemma3:12b`	−5.3

*Mean over the five graded task types, in points. F3 barely changes accuracy on average, but it makes answers far shorter.

Harmless changes, where silence is the right answer:

- **F4 and F6, topic shifts.** Healthy answers replayed under a different question mix, towards long-form (F4) or short-form tasks (F6, the harder one). The model is identical.
- **F7, a rerun.** The healthy model again with fresh seeds.

Hosted and other studies:

- **DeepSeek update (U1).** V4 Flash 0731 against 1017, both pinned to one host (DeepInfra, via OpenRouter) at the same precision. A third condition (Q1) serves the old model at 4-bit from another host.
- **Real traffic.** 3,000 first turns from WildChat (public chatbot logs), answered by both DeepSeek versions. No task types are known.
- **Qwen update.** Qwen3.5 Plus 02-15 against 04-20, both served by Alibaba.
- **The ladder.** The healthy model served differently: 8-bit weights (L1), or temperature 1.0, 1.3 and 1.6 (L2–L4).

Replay. Each condition gets 200 streams of 40 windows. A stream runs healthy answers until an onset at window 10 or later, then switches to the condition's answers to the same questions. We report:

- the share of streams alarmed after the onset (with Wilson intervals);
- latency in windows;
- false alarms per healthy window before the onset;

- alarms on the real held-out windows.

Monitors compared with each other see identical streams, tested with exact McNemar tests.

12. A change of topic looks like a change of model

What we did. Ran the pooled monitor on the local faults and the topic shifts.

What came out. It caught every fault (96–100% of streams within about a window) and also 98% of F4 streams and 100% of F6, where nothing about the model changed. Comparing each task type only with itself (v3) cut F6 alarms to 23% and F4 to 4%, but still fires on about one shift in four and needs task labels.

What it means. The pooled statistics mostly measure *which kinds of questions* a window holds. This is the problem everything after it solves.

13. Re-asking, and why calibration must span days

What we did. Built the paired monitor and first calibrated it on one pair of healthy answer sets.

What came out. It caught every fault but raised 0.21–0.59% false alarms, two to six times its budget. Two healthy rounds generated on different days sit about 10% further apart in embedding distance than the pair used for calibration, though each is just as varied on its own. Lexical diversity alone raised a quarter to a third of the topic-shift alarms.

The fix (pre-registered before it was tried). Generate three healthy rounds on different days, calibrate on all three pairs, keep the largest threshold, and drop lexical diversity. Results after the fix:

study	faults / update caught	short-form topic shift	false alarms
local	100%	0.5%	0.102%
DeepSeek, benchmark	98.5%	0.5%	0.052%
DeepSeek, real traffic	100%	n/a	0.000%
Qwen	98.5%	0.5%	0.000%

What it means. Re-asking removes the topic confusion, including on real traffic, where v3 cannot even be built. A hosted model drifts from day to day, and one round of calibration understates that noise. One honest caveat: the local study sits *on* its budget (0.102%, and 0.092–0.136% across five replays), not under it.

14. One served answer is enough

What we did. Pre-registered the served-answer test, two kernels (characters primary, embedding cosine secondary) and three criteria before computing it on any data:

- **S1:** false alarms at or below 0.1%.
- **S2:** detection within 5 points of Model Equality Testing.
- **S3:** topic-shift alarms no higher than v3's.

What came out (character kernel):

study	caught	short-form topic shift	false alarms	MET caught
local faults	99–100%	0.0%	0.000%	100%
DeepSeek update	95.0%	0.0%	0.000%	100%
DeepSeek on real traffic	94.0%	n/a	0.000%	100%
Qwen update	96.5%	1.0%	0.000%	99.0%

- **No false alarms.** None in 87,716 healthy windows of the primary replay, and one in all 25 replays together.
- **S1 and S3 hold everywhere.**
- **S2 fails once:** on real traffic the test is 6.0 points behind MET.
- **The cosine kernel fails S1 on DeepSeek** (0.234%), through the same day-to-day embedding drift as in §13.

What it means. With a quarter of the production answers it does nearly all the paired monitor does, and it is the steadiest monitor we have. MET, with four production answers, catches 2.5–6 points more of the update streams and alarms a window or two sooner. The theory in §7 says why it falls short exactly where it does (§18).

15. Real updates, in both directions

What came out (% of 200 streams alarmed):

change	accuracy	v1	v3	paired	served	MET
DeepSeek update	+0.2 to +8.6	93.0	94.5	98.5	95.0	100
… on real traffic	n/a	90.0	n/a	100	94.0	100
Qwen update	−0.3 to −26.7	96.0	96.5	98.5	96.5	99.0
DeepSeek at 4-bit	within ±1.5	1.5	3.5	1.0	0.0	0.0

What it means.

- **Both updates are caught within about a window,** whichever way accuracy moved. DeepSeek's update made the model better on every graded task, while Qwen's made it worse (strict JSON −26.7 points, trivia −6.9).
- **The 4-bit host answers almost exactly like the original,** and no monitor flags it. That is a correct silence.
- **v3 broke on Qwen.** It alarmed on a plain rerun of the healthy model in 25% of streams and on 73.5% of topic shifts, at twice its false-alarm budget, through lexical diversity's day-to-day drift. The re-asking monitors held on the same data.

16. The same model, served differently

What we did. Kept the model and prompt and changed only how it is served: 8-bit weights, or temperature 1.0, 1.3 and 1.6. Nothing was recalibrated.

What came out:

monitor	8-bit	T 1.0	T 1.3	T 1.6
v3 (fixed reference)	1.0	50.5	90.5	99.0
paired	96.0	63.5	92.5	99.5
served answer	93.0	43.5	91.5	99.5
MET	96.5	15.5	89.5	98.5
accuracy change	**+2.3**	−0.1	+0.1	+0.4

What it means.

- **Only re-asking sees the precision change.** The 8-bit answers have the same length and mix, so a fixed-reference monitor sees nothing. Compared question by question, the answers differ. This row is the paper's argument in miniature.
- **Higher temperature is caught as answers scramble.** At T 1.0 the served test beats MET by a wide margin, and §7 explains why: the extra variety adds to its score.
- **None of it was harmful.** Accuracy did not fall, and the 8-bit model was 2.3 points better. The monitors answer "did the model change?", not "did it get worse?".

All pre-registered checks hold. The 8-bit round was paused once and resumed after the temperature results were read; the record says so, and notes that this made one hypothesis not blind.

17. Topics on real traffic, found without labels

What we did. Embedded the 3,000 WildChat prompts, clustered them into six topics (k-means, fitted on calibration prompts only), ranked the topics by answer length, and built two harmless shifts that put 70% of a window into the long-answer or the short-answer topics.

monitor	long topics	short topics	false alarms
pooled	33.5%	80.0%	0.204%
paired	1.5%	0.0%	0.010%
served answer	0.0%	0.0%	0.000%
MET	2.0%	0.5%	0.041%

What it means. The topic problem is not an artefact of our benchmark. Real users cause it, and re-asking removes it.

18. Checking the theory

What we did. Estimated every term of §7 for every condition, using the four production and four reference answers per question, and compared the prediction with the measured mean of the served score.

What came out. Prediction and measurement agree on all 14 conditions. Two rows explain the test's behaviour:

condition ($\times 10^{-3}$)	measured score	½·MMD²	½·(K_rr − K_cc)
T 1.0	+5.8	+1.4	+6.5
DeepSeek, real traffic	+29.7	+42.6	−13.1

What it means.

- **At T 1.0 most of the signal is the self-similarity term,** which MET does not see, so the served test beats MET.
- **On real traffic the new model's answers are more alike than the old one's.** The second term is negative and cancels part of the first, which is why the test missed S2 there.
- **The prompt edit (F3) shows the blind spot clearly:** a large MMD², mostly cancelled because the edited model's answers became short and alike.

19. What re-asking costs

What we did. Re-ran the served test with fewer reference answers per question (2, 3 or 4) and on part of each window's questions (all, half or a quarter), recalibrating every setting.

reference calls per served question	4	2	2 (half the questions)	1 (a quarter)
local faults (weakest)	99.0%	95.0%	97.0%	95.0%
DeepSeek update	95.0%	95.0%	93.5%	71.5%
… on real traffic	94.0%	94.0%	85.0%	34.0%
Qwen update	96.5%	96.0%	94.5%	94.0%
8-bit	93.0%	52.0%	82.0%	65.0%

What it means. Big changes are cheap to catch: one reference call per served question still finds the local faults and the Qwen update in 94–97.5% of streams. Subtle changes need the full four. One setting broke the false-alarm budget (Qwen, three references on a quarter of the questions, 0.219%), and it is reported.

20. Slow faults

What we did. Replayed streams where a smaller model answers only part of the traffic after the onset (5%, 10%, 25%, 50% or 100%, or a 30-window ramp). We added two sequential tests (CUSUM and Page–Hinkley) on the served score, calibrated on healthy streams to the same 0.1% budget.

What came out.

bad share of traffic	threshold	CUSUM	Page–Hinkley
F1 at 10%	81.0%	100%	99.0%
F2 at 10%	47.5%	98.0%	92.5%
F1 at 5%	10.0%	22.5%	50.5%
F2 at 5%	1.5%	7.0%	25.0%

No detector raised a single false alarm before the onset. The same CUSUM on the per-family monitor broke its budget three to six times over.

What it means. A model that is wrong on a tenth of traffic is caught within about 14–19 windows. The served score is centred on zero for healthy answers on any questions (§7), so there is no offset for a sequential test to pile up.

21. After the alarm: was it harm?

What we did. After every alarm, ran two follow-ups (designed before they were run):

- **a label-free readout:** how answer length and agreement between samples moved;
- **a graded check:** a person grades 50 or 100 pairs of old and new answers to the same questions, and an exact McNemar test returns harm, improvement or no clear change.

We applied both to the old monitors' alarms and to all 1,833 alarms of the served test.

What came out.

- **No verdict pointed the wrong way.**
- **Faults:** after the smallest model's alarms, the check confirmed harm in 199 of 200 alarms at 100 pairs.
- **Improvements:** after DeepSeek's update it reported improvement, as it did after the 8-bit build.
- **Harmless changes:** after every temperature rung it reported "no clear change", which the oracle confirms.

What it means. An alarm should be read as "go and look", and a small graded check tells what was found. The label-free hint alone can mislead: agreement falls with temperature although accuracy does not.

22. Baselines

On the same local streams:

monitor	F1	F2	F3	F5	F6 (harmless)	false alarms
per-family (v3)	97.0	97.5	100	97.0	23.0	0.053%
paired	100	100	100	100	0.5	0.102%
served answer	100	99.5	99.0	99.0	0.0	0.000%
Model Equality Testing	100	100	100	100	1.0	0.054%
embedding MMD	93.5	86.5	91.5	85.0	94.0	0.587%
LLM judge, Qwen 2.5 14B	89.5	49.0	93.5	3.0	11.5	0.535%
LLM judge, Qwen 2.5 72B	95.5	88.5	92.0	57.0	8.5	0.047%

- **Embedding MMD,** a standard drift test, alarms on 94% of the harmless shift at six times the budget. It answers "did the traffic change?".
- **An LLM judge** that grades each answer from 1 to 10 grades style more than correctness. It hardly sees the older-generation model (3% at 14B, 57% at 72B), whose answers read well but are 5 points less accurate.

23. Earlier checks that still stand

These ran on the fixed-reference monitors and are kept in the paper's appendix:

- **Other deployed models.** With Qwen 2.5 14B, Llama 3.1 8B and Phi-4 14B as the healthy model, the pooled monitor still alarmed on almost every topic shift.
- **Faults on one task type.** Detection follows how visible the change is in that type's answers. Faults confined to intent labels or JSON were caught in at most 3.5% of streams, because a wrong label looks just like a right one.
- **Leakage and seeds.** No question appears in two splits. Ten calibration seeds move local v3 false alarms within 0.017–0.041%.

24. How much the numbers move between replays

What happened. When the 8-bit round joined the ladder, the temperature numbers moved by up to 7 points although their answers were unchanged. Each condition's streams had been seeded by its position in the list, so adding one reshuffled the others.

What we did. Pre-registered a fix: streams are seeded by the condition's name, and every study is replayed five times with thresholds fixed. Replay 0 is primary and gives every number in this brief; replays 1–4 give the spread. No threshold, calibration or answer changed.

What came out.

- **The served test is the steadiest:** one false alarm in 25 replays.
- **The local paired monitor is the least steady:** 0.092–0.136% false alarms.
- **Where no monitor is sure** (T 1.0), detection moves by up to 10 points.

What it means. One replay carries more noise than its binomial interval suggests, so the paper reports the spread beside the headline rates. The rerun also uncovered a real weakness, the local paired monitor's budget, that a lucky single replay had hidden.

25. The hybrid test, the blind test, and the server version

The hybrid test. The served test is weaker when a change makes answers more alike. We pre-registered a fix: on 10% of questions, ask production for one extra answer and use it to estimate the missing term. It kept the false-alarm budget but did not help: on real traffic it caught 93.5% against the served test's 94% and MET's 100%. One extra answer on a tenth of the questions is too noisy, and splitting the budget costs as much as it gains. A pre-registered negative result.

The blind test. Several designs came after earlier ones failed, so we froze everything and tested five changes the monitors had never seen, with no amendment allowed once a result was read.

change	accuracy	v3	paired	served	MET
C1: swapped to Qwen 2.5 14B	−4.5	97.5	100	100	100
C2: swapped to Llama 3.1 8B	−8.6	99.5	100	99.0	100
C3: swapped to Phi-4 14B	+0.3	98.5	100	100	100
C4: a new prompt edit	−0.6	90.0	98.0	98.0	100
C5: answers cut at 256 tokens	−6.3 (maths −32)	94.5	96.5	**0.0**	69.5

Both pre-registered criteria hold: no false alarm in 24,420 windows, and each vendor swap caught in at least 90% of streams. But the served test missed C5, a harmful change. With the same seeds a cut answer is identical to the healthy one up to the cut, and the test compares only the first 1,000 characters (about 250 tokens). The length statistics of v3 and the paired monitor caught it. **Keep a cheap answer-length check beside the served test** (our suggestion, not part of the blind test).

The server version. Ollama updated itself between rounds (0.35.0, 0.40.3, 0.40.9). Two healthy runs across versions look exactly like two runs on one version (MMD^2 -0.30 ± 0.58 against -0.38 ± 0.55, $\times 10^{-3}$), so it did not drive any result. For the blind test we reinstalled 0.40.3 so its new rounds come from one version.

PART IV: WHAT IT MEANS

26. For someone running a model in production

1. **A fixed reference will alarm on ordinary changes in what users ask.** Task labels reduce this but do not remove it, and real traffic has none.
2. **Keep the old model available and re-ask it the live questions.** Comparing answer to answer separates a changed model from changed users, on unlabelled traffic too.
3. **You need no extra production answers.** One served answer per question catches real updates in 94–96.5% of streams with no false alarm in about 88,000 windows.
4. **Re-asking can be cheap.** One reference call per served question is enough for big changes; subtle ones need four. A CUSUM on the served score catches a bad model serving a tenth of traffic with no extra false alarms.
5. **Calibrate on at least three healthy rounds from different days,** and keep a cheap answer-length check beside the served test.
6. **Read an alarm as "something changed, go and look".** Then grade a few dozen answer pairs. Two of the changes we caught made the model better.

27. For research

- **Harmless-shift controls should be standard.** Without one, "100% detection" says nothing.
- **Pre-registration with dated amendments is practical for ML systems work.** It kept us honest when designs failed, which they did several times.
- **Report replay-to-replay variation** next to any detection rate.
- **The served statistic's decomposition** turns a heuristic into something with a stated guarantee and a stated blind spot.

28. Limits

- **Two real updates from two vendors,** one of them also on real traffic.
- **Local models are 4.6–26B open-weight models on one machine,** from one family.
- **Streams are replays of real answers,** not a live deployment.
- **The re-asking monitors need a reference that stays available** (a kept checkpoint or a pinned API version) and reference calls: four per question at full power, fewer for large changes.
- **Several designs came after an earlier one failed:** v3, the three-round calibration and the served test. Each was frozen before the data that judged it, and every revision is recorded.
- **The served test reads only the first 1,000 characters** of each answer, so it missed answers cut at 256 tokens in the blind test.
- **No answer-only monitor sees a fault that leaves the answers looking the same.** The served test is also weakest when a change makes answers more alike.

- **The local paired monitor sits on its budget,** and rates move between replays, by up to 10 points where detection is uncertain.
- **Versions and precisions of hosted models are the hosts' own labels.**

PART V: THE PRACTICAL SIDE

29. The ten constraints

Each blocks one way to produce confidently wrong results; most are enforced by tests.

	constraint	what it prevents
C1	never request `n > 1`; k samples are k rows	lost per-sample seeds
C2	a distinct seed per sample of a question	agreement stuck at 1.0
C3	monitoring code never imports the oracle	labels leaking into a label-free monitor
C4	calibrate on calibration questions only, then freeze	thresholds tuned on the test set
C5	held-out questions only count false alarms	the same, by the back door
C6	unparseable answers are counted, never dropped	hiding the failures we look for
C7	models run in a separate server, pinned by digest	unreproducible calibration
C8	faults share one question subset; only topic shifts change questions	confusing model change with input change
C9	model id, digest and seed logged on every row	an unnoticed model substitution
C10	returned rows reconciled against submitted rows	silently short rounds

The first plan sent generation to a shared batch platform. In the end the local studies ran on one Mac through Ollama, and the hosted studies through OpenRouter, with every request pinned to one host. The batch path is still in the code and tested.

30. What it cost

- **Local generation:** free. Well over 100,000 answers on one laptop (Apple M5 Pro, 48 GB).
- **Analysis:** deterministic and free. Five replays of every study take about 40 minutes.
- **Hosted models:** $26.11 in total, as follows.

item	cost
DeepSeek study	$1.30
real traffic	$2.08
72B judge	$6.81
reference rounds for those two studies	$1.76
Qwen study	$8.26
Qwen reference rounds	$5.90

31. Running it

```
uv sync
uv run pytest -m gate                        # architectural gates, must always pass
scripts/generate_all.sh                      # local rounds and embeddings, in run order
scripts/analyse_all.sh                       # fixed-reference analyses (v1-v3)
scripts/replay_all.sh                        # five replays of every study, all monitors
uv run python scripts/served_extensions.py x1 local   # theory check; x2 cost, x3 slow
faults, x4 triage
python3 scripts/port_papers.py               # rebuild the ICML and NeurIPS texts from ACL
```

Every evaluation verifies its frozen record and refuses to run on a mismatch. Interrupted generation resumes from its checkpoint when the same command is run again. The README has the full run order.

32. The papers

- **ACL version** (`paper/acl/`), "Did the Model Change, or Did the Users? Label-Free Monitoring of Deployed Language Models". Main text 7 of 8 pages, 25 in all; the earlier fixed-reference work is kept in the appendix. This is the submission for ACL Rolling Review (deadline 4 January 2027).
- **ICML and NeurIPS versions** are built from the ACL text by `scripts/port_papers.py` (main text 6 of 8 and 8 of 9 pages).
- **The Responsible NLP checklist** answers are in `paper/responsible_nlp_checklist.md`; the anonymous code archive is `dist/driftwatch-anonymous.zip`.

33. Who did what

- **Dhruv:** generation on both backends, calibration and freezing, the hosted studies, the re-asking monitors, orchestration.
- **Mahak:** windows, the agreement, entropy and embedding signals, detectors.
- **Aditya:** synthetic data, the length and lexical signals, the oracle and grading.
- **Pronnati:** questions and splits, replay, metrics and intervals.

34. What is left before submission

- [ ] **Everyone proofreads the ACL version,** especially their own areas, the pre-registration appendix and the proof (Appendix F). After any edit, run `python3 scripts/port_papers.py`.
- [ ] **Submit to ARR** with the anonymous archive.
- [ ] **Fill in the Responsible NLP checklist** on OpenReview, checking the appendix letters against the final PDF.

Glossary

calibration: measuring what healthy output looks like and setting thresholds from it. **CUSUM / Page–Hinkley:** sequential tests that add up small excursions over windows. **family-wise rate:** the chance that *any* of several statistics fires on a healthy window. **generalised Pareto tail:** a fit to the far tail of a

distribution, so a rare quantile can be read off it. **held-out:** healthy questions used only to count false alarms. **KS statistic:** the largest gap between two empirical distributions. **MMD:** maximum mean discrepancy, a kernel distance between two samples' distributions. **Model Equality Testing (MET):** an MMD test of whether an API serves the same model as a reference, using several answers per prompt. **oracle:** the ground-truth grader; judges monitors, never feeds them. **pre-registration:** fixing the design and pass/fail criteria in a committed file before the data exists. **re-asking:** sending the live questions to a reference model to compare answer with answer. **replay:** a simulated stream built from real answers, with a known onset. **served answer:** the one answer a user actually received. **Šidák correction:** shrinking each statistic's budget so the family-wise rate meets the target. **window:** 200 questions; the unit every statistic is computed over.